

Easier BEACON: Making Automated Fuzzing Simpler

Patrick Elser
Rochester Institute of Technology
Rochester, NY
pte6144@rit.edu

Sidharth Krishna
Rochester Institute of Technology
Rochester, NY
sk9693@g.rit.edu

Abstract – Dynamic software analysis, specifically fuzzing, allows for the automated detection of bugs that can otherwise be hard to uncover through other means. Recent advancements in this space, such as "BEACON: Directed Grey-Box Fuzzing with Provable Path Pruning" propose new techniques to speed up fuzzing, allowing more vulnerabilities to be caught with the same amount of time and computing power. However, it can be hard for developers to implement these solutions into their existing workflows.

As such, we propose Easier BEACON: Making Automated Fuzzing Simpler. Easier BEACON integrates with developers' existing CI/CD pipelines to automatically and precisely fuzz newly pushed code within a comprehensive, easy to use docker image. We believe this will make it possible for more projects to implement modern fuzzing, so they can detect vulnerabilities on their own without relying on external security researchers.

I. Introduction

A. Software Analysis

Software always contains some number of bugs. Whether it is because programmers rush,

fail to consider certain edge cases, or simply make mistakes, every software project of a big enough size will have some bugs within their source code. As such, for a good security in depth approach, it is important to not only try to prevent bugs from being written in the first place, but also proactively seek them out and squash them.

The field of software analysis seeks to do just that. By analyzing software with a variety of tools, analyzers are able to find these bugs and use this information to remedy them. This can happen at nearly any stage of the development lifecycle, from a tool integrated in an IDE flagging issues as they are written to reproducing crashes that have been reported in production code. Regardless of when the analysis happens, though, it can provide invaluable assistance in increasing the security of our software.

One of the most common sets of groupings software analysis techniques are divided into are static and dynamic. The distinction between them being whether or not the code in question is executed in any way. When people think of analyzing code, they often think of static analysis techniques that read in source code and find patterns that align with known vulnerabilities. While static analysis is undoubtedly important and has many key advantages over dynamic analysis, in this paper

we will be focusing on a form of dynamic analysis.

B. Dynamic Analysis

Dynamic Analysis, by contrast, involves actually running the code in question, usually within some form of virtualized environment. Developers often perform one of the simplest forms of dynamic analysis, running a program in a debugger. While this is not simple from an implementation and technical perspective, most programmers will learn how to use a debugger in their early coursework, and as such, most are able to use visual debuggers easily.

However, debugging a program manually is a time consuming process. While it makes sense if a developer is trying to learn more about a known issue in a particular section of their code, it isn't helpful for finding new vulnerabilities or ensuring code is bug free. As such, more automated forms of dynamic analysis, while foreign to many developers, can fill a role that simply can't be performed by standard debugging.

C. Continuous Integration / Continuous Delivery (CI/CD)

CI/CD is a common term to most developers writing code today. It refers to automated actions, sometimes called pipelines or workflows, being performed as code is written. For instance, when a developer pushes their changes to git, jobs automatically run that perform actions on their code. As the name suggests, CICD can perform integration actions, such as compiling code or building packages and docker containers, as well as deliver code, pushing the packages to repos and even logging into production systems and pulling down the updates. That said, a variety of other uses for these automated actions have been put in place, such as automatically linting developer code to align with project style guidelines.

In particular, we are concerned with the use of CI/CD pipelines to perform automated software analysis on newly pushed code, identifying vulnerabilities quickly and quietly, without wasting developer time. Relying on analysis to be manually run will generally result in checks being spotty at best, and will most likely just end up being added to the mountain of tech debt most development teams are buried under.

D. Gitlab CI/CD

In particular, we have focused on the CI/CD within the Gitlab platform. While not as popular as Github, Gitlab boasts a number of advantages making it a good choice for many large projects. For instance, the FreeDesktop project, which develops a number of core parts of graphical stack for linux systems, uses a self-hosted instance of Gitlab. If companies want to host their git repos on their own servers, rather than pushing their code to Microsoft, Gitlab is arguably the most mature, and certainly one of the most common, choices.

In our case, we also chose Gitlab due to the ability to self-host. As we will discuss further, our work is somewhat resource intensive, so being able to run it on local hardware is an important step to prevent fees from being required, or otherwise being severely limited with low resource environments slowing us down.

Gitlab CI/CD jobs, by default, are run within docker containers. Project owners are able to specify containers and scripts to run within these containers, and are then able to save the outputs or push them directly to package and container repos built directly into Gitlab itself. Specifying these jobs is done within a yaml file in the root of the project directory, in a file called `.gitlab-ci.yml`.

II. Background

A. Fuzzing

Fuzzing is a dynamic testing technique that automatically generates and executes large volumes of program inputs to discover bugs, crashes, and security vulnerabilities. Compared to traditional static analysis, fuzzing does not require deep program understanding or formal specifications, which makes it highly scalable for large real-world software systems [12]. Early fuzzers relied on purely random input generation, but this led to limited ability to penetrate deeper program logic, especially when reaching vulnerable code that required satisfying complex conditions or multi-stage predicates.

Coverage-guided fuzzing, popularized by American Fuzzy Lop (AFL) [12], introduced feedback mechanisms that prioritize inputs increasing code coverage. AFL tracks basic block transitions to guide mutations more intelligently, significantly outperforming purely random approaches. Later hybrid fuzzers such as SAGE [7], Angora [9], and ProFuzzer [10] augmented fuzzing with symbolic execution or type-aware inference to handle challenging constraints. However, these approaches often incur substantial computational overhead and struggle with scalability when used against large or complex codebases. Overall, fuzzing provides a practical baseline for vulnerability discovery, but becomes inefficient when the goal is to reach a specific program point, motivating the evolution toward directed fuzzing.

B. Directed Fuzzing

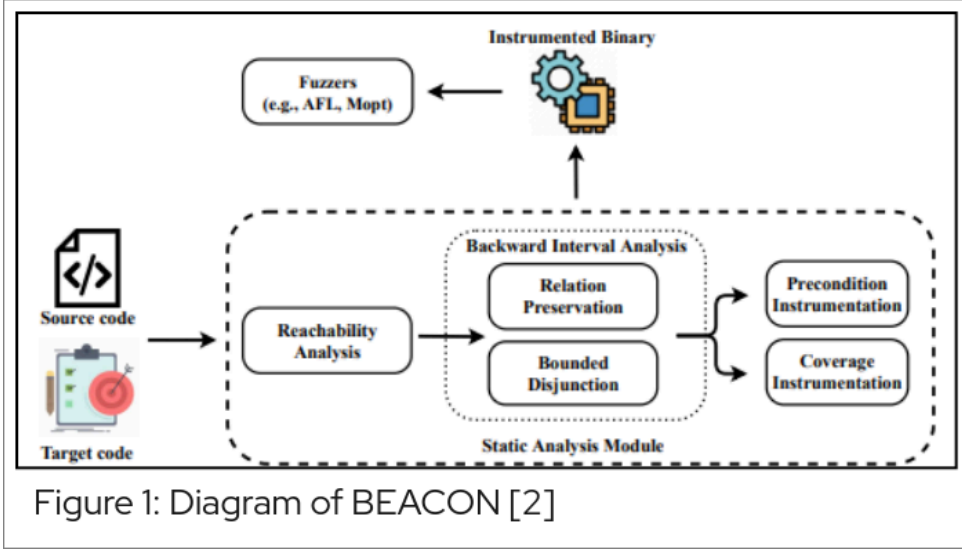
Directed Greybox Fuzzing (DGF) focuses on reaching specific target locations in a program—such as crash sites, patched regions, or security-critical routines—rather than maximizing global coverage. This makes DGF especially useful for crash reproduction, patch

testing, and targeted vulnerability exploration [1][2].

AFLGo is one of the earliest directed fuzzers, introducing a distance-based guidance metric computed over control-flow and call-graph paths [1]. By prioritizing seeds with shorter graph distances to the target, AFLGo biases the fuzzer toward relevant program regions during its simulated annealing-style exploration. Although effective as a baseline, AFLGo’s purely distance-based prioritization does not prune infeasible paths, causing it to waste time mutating seeds that can never reach the target.

Subsequent systems improved guidance precision. Hawkeye refined distance estimation by incorporating both fine-grained and coarse-grained information, including enhanced call-graph modeling and improved mutation ranking [11]. Constraint-Guided Directed Greybox Fuzzing (CAFL) [3] introduced an ordered-constraint model that captures intermediate program points required for reaching a bug, improving multi-stage crash reproduction. DAFL [4] incorporated data-dependency analysis to focus fuzzing on paths influencing target-relevant variables, while HyperGo [5] integrated probabilistic symbolic reasoning to improve target reachability.

Despite these advancements, most directed fuzzing frameworks rely heavily on heuristic distance metrics, probabilistic estimation, or mutation prioritization. They do not provide provable guarantees that infeasible paths will be rejected early, which limits their efficiency on large or constraint-heavy codebases. BEACON directly addresses this gap by introducing static precondition inference to prune infeasible paths before or during fuzzing.



C. BEACON

BEACON represents a significant advancement in directed fuzzing by introducing provable path pruning through static precondition inference [2]. Unlike earlier directed fuzzers that rely mainly on heuristic or distance-based guidance, BEACON uses backward interval analysis over LLVM bitcode to compute necessary conditions for reaching the target location. By propagating constraints from the target back to the entry point, BEACON determines value ranges that key variables must satisfy along feasible paths.

This static analysis produces a mapping of required numeric conditions, which are then inserted into the program as lightweight runtime assertions. During fuzzing, if an input leads to a program state violating these inferred constraints, execution is terminated early—eliminating infeasible paths that would otherwise consume time and mutation effort. BEACON enhances precision through techniques such as relationship preservation (retaining linear relationships between variables) and bounded disjunction (maintaining multiple abstract states at merge points to avoid over-approximation) [2]. These features allow

BEACON to retain strong soundness guarantees while preventing the imprecision typical of classical interval analysis.

In BEACON’s architecture (Figure 1), the static analysis module performs reachability analysis followed by backward interval analysis, enhanced through relation preservation and bounded disjunction to maintain precision. The inferred preconditions are then injected into the binary through selective instrumentation, producing an instrumented executable that any greybox fuzzer (e.g., AFL, MOpt) can use while automatically pruning infeasible paths during execution.

Empirical evaluations show that BEACON prunes over 80% of infeasible paths and achieves significant speedups compared to AFLGo, Hawkeye, and state-of-the-art directed fuzzers [2]. As a fuzzer-agnostic enhancement, BEACON can be integrated with AFL, AFLGo, MOpt, and other fuzzers, consistently improving their performance by reducing wasted exploration time. In the broader directed fuzzing landscape, BEACON marks the transition from heuristic guidance strategies toward static-analysis-augmented fuzzing frameworks

capable of providing provable pruning and more efficient target-directed execution.

III. Areas for Improvement

A. Using BEACON

While BEACON is a great tool for directed greybox fuzzing, it is somewhat onerous to use. In their paper, Huang et al. only link to a docker image containing precompiled binaries for their proof of concept, rather than source code. This container, in turn, is not well documented, but does link to a new Github repository uploaded after their paper was published. Within this repo is the source code necessary to run BEACON, as well as documentation and steps on how to do so.

In order to use BEACON, an analyst would first need to download the necessary libraries and tools. One of these libraries, libtinfo5, is no longer available in the standard Ubuntu repositories, as it went end of life in 2023. As such, this package must be manually pulled from security.ubuntu.com and applied with the dpkg command. In addition, an old version of the clang compiler, compatible with LLVM 4.0.0, must be downloaded, as well as the source code for the SVF project. As such, a developer cannot have clang installed on their computer already, as it would conflict with the older downloaded version.

Once these tools are installed, the SVF project has to be built, followed by building the two sections of BEACON, the precondition inference engine and the instrumentation tool. Only once these two tools are built can a developer begin instrumenting their project.

To do this, a developer would compile their program with wllvm, a python3 project for compiling whole programs with LLVM intermediate representation (LLVM IR). When doing this, the developer should specify the downloaded clang as the designated compiler.

Then, they can use extract-bc, another part of wllvm, to get the bitcode file that BEACON instruments.

From here, a developer will run the BEACON precondition inference engine, which will output multiple text files detailing its results along with a slightly modified bitcode file, though it has not yet instrumented it. Then, the developer will run the new bitcode file through the instrumentation tool, providing it two of the text files outputted by the previous state. With the newly instrumented bitcode file, a developer can finally use clang directly to compile it back into executable code, complete with instrumentation.

It should be relatively obvious that this is not an ideal solution for developers to follow. While the provided docker container does cut down on the steps, this container still expects to be given bitcode, meaning that the older version of clang and wllvm will still be needed.

This is not to say that the provided implementation of BEACON is bad. The proof of concept does work well for validating the theories of the authors and proving that terminating these branches early does noticeably improve performance, which was the primary goal of their implementation. It also is useful for security researchers, who may use a single build instance of BEACON to scan many open source projects, reporting back their findings. This project also makes a good base for future research. However, as is, BEACON is not reasonably usable by developers and other members of teams responsible for securing projects.

B. Specifying the Target Line

Another issue we have with the provided implementation is the process of specifying the target line. The current way this is done is by creating a file in which the only contents are the target file and line, separated by a color. For example, a 27th line of the main file of a

program may be written as `main.c:27`. The file path, then, is passed into the precondition inference engine when using BEACON to instrument a target binary.

This is not a very flexible approach. One of the arguments for greybox fuzzing itself is to target suspicious or bug-prone lines of code, such as `free()` calls. As program files are modified, these lines of code will move around. Perhaps a new include is added to a c file for new capabilities, maybe a function is moved above or below another. In either case, the line numbers of code can easily shift, forcing programmers to play a game of cat and mouse keeping the target line file up to date.

In an automated solution within CI/CD, as we propose in the next section, this is an even more severe problem. One common item seen in CI/CD pipelines is linters, which edit source code files to conform to style guidelines. A linter might be set to ensure that there is exactly one blank line between the end of one function and the beginning of the next. If a programmer pushes new code and specifies a line for targeted fuzzing, but the linter gets rid of an accidental second blank line between functions, an incorrect line will be selected. This could cause BEACON to crash on an invalid line or even target unrelated code, giving developers a false sense of security that their new code has been checked.

IV. Proposed Solution

A. Simplifying Usage

We agree with the authors of BEACON that a docker container is the logical way to distribute BEACON. Docker containers allow for software to be easily distributed to many systems, both workstations and servers, and be run in a repeatable manner. Compared to traditional packages, which would need to be recompiled and repackaged with different OSs,

docker containers are a single solution. As they bring their own libraries, there is no concern about version mismatches or packages disappearing from distribution repositories.

Compared to full virtual machines, though, docker containers are considerably lighter weight. They can spin up and spin down faster, use less disk space, and consume less resources. They do lose some platform agnosticism, as Windows cannot easily run docker containers that rely on the Linux kernel. In addition, traditional virtual machines are even more independent from their host systems, which would be ideal. In all, though, the advantages of lighter, easier to use containers make them the best choice when the goal is to simplify usage.

Compared to the image distributed by BEACON, we chose to make one major change. Rather than just packaging BEACON, which requires developers to generate their bitcode separate from the container, we decided to package all the tools necessary to go from source code to instrumented binary directly in our image. We also packaged a compatible version of AFL in the image, allowing the single container to be used for both building and fuzzing, though in practice these two steps should usually be separate containers from our single image.

B. Dynamic Target Line Selection

Dynamic target line selection is a practical mechanism for identifying and isolating specific program locations that a directed fuzzer should attempt to reach. In real-world vulnerability analysis and reproduction workflows, analysts often need to mark certain code regions as “targets”—for example, suspicious blocks, patch-introduced logic, or locations previously associated with crashes. However, these target sites may change between experiments, or multiple experiments may require switching between different target contexts. A static,

hard-coded target list therefore becomes inflexible.

To address this, dynamic target selection allows the analyst to embed lightweight markers directly within the program source code, which can then be automatically extracted prior to fuzzing. Tools or scripts scan the source file, locate these marker comments, and output file-line pairs in a machine-readable format. These extracted targets can then be supplied to directed fuzzers (such as AFLGo or BEACON) to constrain execution toward the designated code region.

In this workflow, a source file is annotated with a simple marker- `//?BEACON`. A lightweight scanning utility then processes the file, searches for all occurrences of this marker, and records the corresponding file name and line numbers. The previously provided C++ utility serves this purpose: it accepts a file path, scans for a target comment, and outputs results in the format required by BEACON. (filename:linenumber). This allows analysts to adjust target selection simply by editing source-level markers, without modifying analysis configuration files or build scripts.

Dynamic target line selection is particularly useful in directed fuzzing workflows that require iterative exploration, patch validation, or multi-stage debugging. For example, when investigating a crash, the analyst can first mark the crashing line, fuzz to reproduce it, then mark upstream conditions or reverse-engineered logic to guide fuzzing toward intermediate states. Because target discovery is automated through marker detection, the overall process becomes flexible and repeatable, supporting a wide range of experiments without changes to the fuzzer itself.

C. Automation and Integration with CI/CD

Docker containers naturally are very easy to integrate into Gitlab CI/CD. By default, workflows will run within a default docker

image; we found the default image to be a ruby image. However, it is very easy to specify an alternative image, and most developers will do so unless they happen to be testing a ruby image. As such, the template we provide will use our single build image, mentioned in Section IV.A.

Despite using one image, our template should ideally split the build and test phases out separately, to better integrate into existing setups. Gitlab only allows for defining a single CI file, which in turn can be as long and include as many steps as is needed. These steps, as long as they are specified as being in the same stage, can run simultaneously, so there is little concern of the fuzzing process slowing down other, unrelated CI/CD jobs.

In addition, we want our template to output both the uninstrumented and instrumented binaries to the user, allowing them to be downloaded and tested. In the event that AFL and BEACON find a crash, whoever investigates the issue will first want to reproduce it, and having the exact binaries in question easily accessible will make this process much easier.

V. Implementation and Evaluation

A. The Docker Image

Our docker image is publicly available on dockerhub.com/r/patrickelser/beacon. We walk through the container here, but if you wish to see the individual steps within the dockerfile, follow the above link and go to tags and select the latest tag. This should show the steps used to build the image. In addition, our dockerfile is also available for review or modification on github at github.com/pte6144-rit/Beacon_CSEC_759_RIT.

For compatibility with BEACON, we chose to use the ubuntu image as our base, simply selecting the latest version available at the time of building. From here, we install BEACON's dependencies and follow the steps to pull and

use the version of clang with LLVM 4.0.0. This

inference engine and the instrumentation tool.

```
stages:
  - build
  - test

build-1:
  stage: build
  image:
    name: patrickelser/beacon
    entrypoint: [""]
  script:
    - /bin/bash /beacon/beacon.sh <path-to-source-file>
  artifacts:
    paths:
      - ./<uninstrumented-binary-name>
      - ./<instrumented-binary>

fuzz:
  stage: test
  image:
    name: patrickelser/beacon
    entrypoint: [""]
  script:
    - echo "Beginning Fuzzing"
    - echo core >/proc/sys/kernel/core_pattern
    - /beacon/Fuzzer/afl-fuzz -i .fuzzin -o fuzz_out <instrumented-binary>
    - echo "Ending Fuzzing"
  artifacts:
    paths:
      - ./fuzz_out
```

Figure 2: CI/CD File Template

involves installing python3 and using pip to install wllvm, as well as pulling down the package for libtinfo5 directly.

Once these tools are installed, we clone and build SVF directly from their git repository. Instead of pulling BEACON itself from git, seeing as our dockerfile is in a fork of BEACON, we simply add the necessary folder from our fork directly into the container. After adding them, we build both the precondition

After this, we build our line number selection tool as described in Section V.B.

Finally, a script that performs all necessary building and instrumentation steps is added to the repository. This allows for the CI/CD file to be much shorter and approachable, hiding implementation details away from developers who don't need to fully understand how the instrumented binary is built.

Performing all the build steps in a single script also greatly reduces the number of places where things can go wrong or steps could be skipped. As mentioned in Section III, the process of using beacon is rather complex, with multiple utilities needed to build the bitcode file, multiple utilities needed to instrument the bitcode file, and yet another utility to finally compile the bitcode into a runnable binary. Unifying this into a single, documented script is part of our contribution to this project.

B. Line Number Selection

To find these target lines, we developed a simple C program, available in the aforementioned git repository. This program is able to scan for and output the file name and line number it finds the target line comment on.

This binary is compiled during the container build process, along with the two BEACON binaries. Its output is then incorporated into the main beacon.sh script, which redirects it into a file. This file is passed then to BEACON's precondition inference engine as normal, using the same format as in the original proof of concept.

C. Gitlab Pipelines

In Figure 2 above, we show a template CI/CD yaml file. In our implementation, the process of using BEACON is split into two distinct parts, the build job and the fuzz job. To better align these two jobs with many teams' existing infrastructure, we placed these two jobs into common stages 'build' and 'test'. This allows the BEACON build process to be run parallel to any other build processes and the AFL fuzzing steps parallel to any other tests. However, it is still ensured that BEACON will finish and upload its compiled binaries before AFL is run.

The script sections in both jobs are succinct, allowing for easier implementation. The first job

only has a single line, calling the aforementioned script which handles turning source code into an instrumented binary. The fuzzing job requires two lines, one to set the core dump pattern so AFL can properly read crashes and another to run AFL. The other two echos are merely for logging purposes.

The final part of the pipeline configuration is the artifact sections. Artifacts are uploaded to Gitlab and can be downloaded by any developer with sufficient permissions, allowing the results from AFL, such as the inputs associated with crashes, to be easily downloaded. The compiled binaries, both the instrumented and the uninstrumented, can also be downloaded in this manner.

The two stages can easily be copied directly into an existing Gitlab project's gitlab-ci.yml file, allowing existing projects to begin using BEACON for fuzzing more easily. That said, it is recommended that developers add restrictions to when BEACON is run, such as only on certain branches, to avoid wasting resources running full fuzzing on any partial development commit.

VI. Remaining Issues

A. CMake Compatibility

While we were able to execute CMake from within the docker container, we unfortunately ran out of time to properly integrate it into the script. All build tools necessary for CMake are already present within the container, they simply need to be set up to be able to compile the bitcode for the project. After the bitcode is compiled, all further steps are the same for both multiline projects and single file projects as already implemented.

B. Multi Line Support

In a practical development setting there may be multiple target locations-such as patch sites

or vulnerability candidates, or intermediary states required to reach a deeper bug. For these scenarios, the target line extraction program must support multiple tags in the same file.

Multi-line support allows programmers to place more than one “`/?BEACON`” tag anywhere in the source file. The scanning utility processes the entire file to give a separate output for each tag in the `filename:linenumber` format. These entries can then be passed to the fuzzers. This capability enables a flexible and scalable workflow, enhancing repeatability and adaptability in directed fuzzing scenarios involving complex programs where multiple execution points contribute to trigger conditions of a vulnerability.

In addition, it may make sense to expand the comment syntax to support independent sets of lines. For instance, a programmer might be concerned about use after free vulnerabilities with the variable `x` and the variable `y`. To target this, they could put specific comments, like `/?BEACON(use_after_free_x)` for lines that use `x` and `/?BEACON(use_after_free_y)` for lines that involve `y`. This would allow for both sets of variables to be fuzzed in sequence, allowing multiple `x` lines to be targeted and multiple `y` lines to be targeted without them interfering with each other.

VII. Future Work

As might be expected, the primary place where future work is needed is to patch the aforementioned issues. However, there is still room for more improvement beyond this.

To make configuration simpler, the various variables in our template could be abstracted into a configuration file, such as `.beacon.yml`. In this case, developers could simply copy the CI/CD pipelines as is, only needing to instead modify a more straightforward configuration file. While in the current state this level of abstraction is not needed, it may be more useful

in more complex projects with large `.gitlab-ci.yml` files and as more features, like CMake support, are added.

Another place where our solution could be improved is in forward compatibility. Currently, we focus heavily on BEACON, but recent works have built on BEACON and will continue to do so for the foreseeable future. Standardizing a container format for various instrumentation implementations and fuzzers could allow developers to move to newer technologies and techniques by simply changing image names, without needing to rework any other parts of the pipeline configuration.

Finally, this solution could be imported to other git platforms, such as Github. While we did not want to pay for the extra computing power needed, many companies may choose to, and would want to be able to use features like the ones we have proposed to fuzz their projects. As such, implementations of the pipelines, or workflows as Github calls them, that are able to reuse the same docker images would be useful as a future work.

VIII. Conclusion

We simplified the use of BEACON, a directed greybox fuzzer, by offering a ready to use CI/CD pipeline configuration that cleanly separates the build and test stages. This involved creating a unified docker image that bundles all complex dependencies like LLVM and BEACON, to streamline the build process. Then implementing a dynamic target line selector program which uses tags and bundling all this into a Gitlab template, transforming BEACON into an accessible, automated tool.

These tools significantly reduce barriers for developers who wish to use recent fuzzing techniques, like BEACON to secure their code. With automated techniques like this, bugs and CVEs can be found earlier and patched before

they are exploited, improving our overall security.

REFERENCES

- [1] Pham, V.-T., Böhme, M., Roychoudhury, A. “AFLGo: Directed Greybox Fuzzing,” CCS, 2017.
- [2] Huang, H., Guo, Y., Shi, Q., Yao, P., Wu, R., Zhang, C. “BEACON: Directed Grey-Box Fuzzing with Provable Path Pruning,” IEEE S&P, 2022/2023.
- [3] Lee, G., Lee, B., Han, B. “Constraint-Guided Directed Greybox Fuzzing,” USENIX Security, 2021.
- [4] Kim, T.-E., et al. “DAFL: Directed Greybox Fuzzing Guided by Data Dependency,” USENIX Security, 2023.
- [5] Lin, Y., et al. “HyperGo: Probability-Based Directed Hybrid Fuzzing,” 2023.
- [7] Godefroid, P., et al. “SAGE: Whitebox Fuzzing for Security Testing,” Communications of the ACM, 2012.
- [8] Rawat, S., et al. “VUzzer: Application-Aware Evolutionary Fuzzing,” NDSS, 2017.
- [9] Chen, K., et al. “Angora: Efficient Fuzzing by Principled Search,” IEEE S&P, 2018.
- [10] You, W., et al. “ProFuzzer: On-the-fly Input Type Probing,” IEEE S&P, 2019.
- [11] Wang, J., et al. “Hawkeye: Towards a Desired Directed Greybox Fuzzer,” CCS, 2019.
- [12] Zalewski, M. “American Fuzzy Lop (AFL),” 2014

Appendix I. Example Gitlab Logs (Truncated)

```
1 Running with gitlab-runner 18.4.0-pre.246.g71914659 (71914659)
2   on green-6.saas-linux-small-amd64.runners-manager.gitlab.com/default YKxHnyexq, system ID
3 Preparing the "docker+machine" executor
4 Using Docker executor with image patrickelser/beacon:rebuild ...
5 Using effective pull policy of [always] for container patrickelser/beacon:rebuild
6 Pulling docker image patrickelser/beacon:rebuild ...
7 Using docker image sha256:fa6cbc9b4cccec71a82ec1288d3423df2d4d6c30b7194bd4ca818e8bf2d4972f-
8 Preparing environment
9 Using effective pull policy of [always] for container sha256:d4862e94a39233256d99974dbab59b3
10 Running on runner-ykxhnyexq-project-76240026-concurrent-0 via runner-ykxhnyexq-s-l-s-amd64-
11 Getting source from Git repository
12 Gitaly correlation ID: e000d9a2949343ebb353266b926f65aa
13 Fetching changes with git depth set to 20...
14 Initialized empty Git repository in /builds/pte6144/beacon_testcase_1/.git/
15 Created fresh repository.
16 Checking out 6786dd12 as detached HEAD (ref is master)...
17 Skipping Git submodules setup
18 $ git remote set-url origin "${CI_REPOSITORY_URL}" || echo 'Not a git repository; skipping'
19 Downloading artifacts
20 Downloading artifacts for build=1 (12312668274)...
21 Downloading artifacts from coordinator... ok correlation_id=7a4eed62f9d74d879d923a1c
22 Executing "step_script" stage of the job script
23 Using effective pull policy of [always] for container patrickelser/beacon:rebuild
24 Using docker image sha256:fa6cbc9b4cccec71a82ec1288d3423df2d4d6c30b7194bd4ca818e8bf2d4972f-
25 $ echo "Beginning Fuzzing"
26 Beginning Fuzzing
27 $ echo core >/proc/sys/kernel/core_pattern
28 $ /beacon/Fuzzer/afl-fuzz -i ./fuzzin -o ./fuzz_out ./standalone-ins
29 aflgo (yeah!) 2.52b
30 [+] Running with EXP schedule and time-to-exploitation set to 10 minutes
31 [+] Looks like we're not running on a tty, so I'll be a bit less verbose.
32 [+] You have 2 CPU cores and 1 runnable tasks (utilization: 50%).
33 [+] Try parallel jobs - see docs/parallel_fuzzing.txt.
34 [+] Checking CPU core loadout...
35 [+] Found a free CPU core, binding to #0.
36 [+] Checking core_pattern...
37 [+] Setting up output directories...
38 [+] Scanning './fuzzin'...
39 [+] No auto-generated dictionary tokens to reuse.
40 [+] Creating hard links for all input files...
41 [+] Validating target binary...
42 [+] Attempting dry run with 'id:000000,orig:example'...
43 [+] Spinning up the fork server...
44 [+] All right - fork server is up.
45   len = 10, map size = 1, exec speed = 267 us
46 [!] WARNING: No new instrumentation output, test case may be useless.
47 [+] All test cases processed.
48 [!] WARNING: Some test cases look useless. Consider using a smaller set.
49 [+] Here are some useful stats:
50   Test case count : 1 favored, 0 variable, 1 total
51   Bitmap range : 1 to 1 bits (average: 1.00 bits)
52   Exec timing : 267 to 267 us (average: 267 us)
53 [+] No -t option specified, so I'll use exec timeout of 20 ms.
54 [+] All set and ready to roll!
55 [+] Entering queue cycle 1.
56 [+] Fuzzing test case #0 (1 total, 0 uniq crashes found)...
57 [+] Entering queue cycle 2.
58 [+] Fuzzing test case #0 (1 total, 0 uniq crashes found)...
59 [+] Entering queue cycle 3.
60 [+] Fuzzing test case #0 (1 total, 0 uniq crashes found)...
61 [+] Entering queue cycle 4.
62 [+] Fuzzing test case #0 (1 total, 0 uniq crashes found)...
```

```

1 Running with gitlab-runner 18.4.0~pre.246.g71914659 (71914659)
2   on green-2.saas-linux-small-amd64.runners-manager.gitlab.com/default ns46NMmJT, system ID: s
✓ 3 Preparing the "docker+machine" executor
4 Using Docker executor with image patrickelser/beacon:rebuild ...
5 Using effective pull policy of [always] for container patrickelser/beacon:rebuild
6 Pulling docker image patrickelser/beacon:rebuild ...
7 Using docker image sha256:fa6cbc9b4cccec71a82ec1288d3423df2d4d6c30b7194bd4ca818e8bf2d4972f for
✓ 8 Preparing environment
9 Using effective pull policy of [always] for container sha256:a15180be7c53e13092d3f404eede2b5a5:
10 Running on runner-ns46nmmjt-project-76240026-concurrent-0 via runner-ns46nmmjt-s-l-s-amd64-176
✓ 11 Getting source from Git repository
12 Gitaly correlation ID: 25961c4c864c4cd78f5e96f0d3db8d11
13 Fetching changes with git depth set to 20...
14 Initialized empty Git repository in /builds/pte6144/beacon_testcase_1/.git/
15 Created fresh repository.
16 Checking out 6786dd12 as detached HEAD (ref is master)...
17 Skipping Git submodules setup
18 $ git remote set-url origin "${CI_REPOSITORY_URL}" || echo 'Not a git repository; skipping'
✓ 19 Executing "step_script" stage of the job script
20 Using effective pull policy of [always] for container patrickelser/beacon:rebuild
21 Using docker image sha256:fa6cbc9b4cccec71a82ec1288d3423df2d4d6c30b7194bd4ca818e8bf2d4972f for
22 $ /bin/bash /beacon/beacon.sh src/standalone.c
23 ---Start WLLVM---
24 ---START EXTRACTION---
25 ---START PRECONDINFER---
26 ****CallGraph SCC Stat****
27 ##### (program : )#####
28 -----
29 TotalNode      8
30 TotalCycle     0
31 MaxNodeInCycle 0
32 NodeInCycle    0
33 TotalEdge      9
34 CalRetPairInCycle 0
35 #####
36 ****Andersen Pointer Analysis Statistics****
37 ##### (program : )#####

```

```

187 ---START COMPILE---
188 ---Finish COMPILE---
✓ 189 Uploading artifacts for successful job
190 Uploading artifacts...
191 ./standalone: found 1 matching artifact files and directories
192 ./standalone-ins: found 1 matching artifact files and directories
193 Uploading artifacts as "archive" to coordinator... 201 Created co
✓ 194 Cleaning up project directory and file based variables
195 Job succeeded

```